

Grouping and Aggregation in Pandas

Introduction

In data analysis, we often need to **group similar records** together and calculate summary statistics such as total sales, average marks, highest salary, or the number of employees in each department.

Pandas provides the powerful **GroupBy** functionality, which follows the **Split → Apply → Combine** strategy:

- **Split** the data into groups.
- **Apply** one or more functions to each group.
- **Combine** the results into a new DataFrame or Series.

Grouping and aggregation are among the most widely used operations in Data Science, Business Intelligence, Finance, Healthcare, Marketing, and Machine Learning.

Learning Objectives

By the end of this chapter, you will be able to:

- Understand GroupBy operations.
 - Group data by one or more columns.
 - Perform aggregation functions.
 - Apply multiple aggregation functions.
 - Rename aggregated columns.
 - Use custom aggregation.
 - Filter grouped data.
 - Transform grouped data.
 - Solve real-world grouping problems.
-

Importing Pandas

```
import pandas as pd
```

Sample Dataset

We will use the following employee dataset throughout this chapter.

```
import pandas as pd
```

```
data = {  
    "Employee":["Amit","Rahul","Priya","Neha","Karan","Anjali","Rohit","Sneha"],  
    "Department":["IT","HR","IT","HR","Sales","Sales","IT","HR"],  
    "City":["Mumbai","Delhi","Mumbai","Delhi","Pune","Pune","Delhi","Mumbai"],  
    "Salary":[70000,50000,80000,55000,45000,48000,75000,60000],  
    "Experience":[5,2,6,3,2,4,5,7]  
}
```

```
df = pd.DataFrame(data)
```

```
print(df)
```

Output

Employee	Department	City	Salary	Experience
Amit	IT	Mumbai	70000	5
Rahul	HR	Delhi	50000	2
Priya	IT	Mumbai	80000	6
Neha	HR	Delhi	55000	3
Karan	Sales	Pune	45000	2
Anjali	Sales	Pune	48000	4

Rohit	IT	Delhi	75000	5
Sneha	HR	Mumbai	60000	7

What is Grouping?

Grouping means dividing the dataset into smaller groups based on common values.

Example:

Department-wise employees

IT

Amit

Priya

Rohit

HR

Rahul

Neha

Sneha

Sales

Karan

Anjali

GroupBy Syntax

```
df.groupby("Column_Name")
```

Example: Group by Department

```
group = df.groupby("Department")
```

```
print(group)
```

Output

```
<pandas.core.groupby.generic.DataFrameGroupBy object>
```

The `groupby()` method creates a **GroupBy object**. To obtain useful results, apply aggregation functions.

Viewing Groups

```
print(group.groups)
```

Output

```
{  
'HR':[1,3,7],  
'IT':[0,2,6],  
'Sales':[4,5]  
}
```

Iterating Through Groups

```
for department, data in df.groupby("Department"):
```

```
    print(department)
```

```
    print(data)
```

Aggregation Functions

Aggregation means summarizing data.

Common aggregation functions include:

- `sum()`
 - `mean()`
 - `max()`
 - `min()`
 - `count()`
 - `size()`
 - `median()`
 - `std()`
 - `var()`
 - `first()`
 - `last()`
-

Sum

Find the total salary of each department.

```
df.groupby("Department")["Salary"].sum()
```

Output

Department	Salary
HR	165000
IT	225000
Sales	93000

Mean

Average salary by department.

```
df.groupby("Department")["Salary"].mean()
```

Output

Department	Salary
HR	55000
IT	75000
Sales	46500

Count

Number of employees in each department.

```
df.groupby("Department")["Employee"].count()
```

Output

Department	Employee
HR	3
IT	3
Sales	2

Size

```
df.groupby("Department").size()
```

Output

Department	Size
HR	3

IT	3
----	---

Sales	2
-------	---

Difference:

- `count()` ignores missing values.
 - `size()` counts all rows.
-

Maximum Salary

```
df.groupby("Department")["Salary"].max()
```

Output

Department	Salary
HR	60000
IT	80000
Sales	48000

Minimum Salary

```
df.groupby("Department")["Salary"].min()
```

Median Salary

```
df.groupby("Department")["Salary"].median()
```

Standard Deviation

```
df.groupby("Department")["Salary"].std()
```

Variance

```
df.groupby("Department")["Salary"].var()
```

Multiple Aggregations

We can calculate several statistics simultaneously.

```
df.groupby("Department")["Salary"].agg(["sum","mean","min","max"])
```

Output

Department	Sum	Mean	Min	Max
HR	165000	55000	50000	60000
IT	225000	75000	70000	80000
Sales	93000	46500	45000	48000

Custom Column Names

```
df.groupby("Department").agg(  
    Total_Salary=("Salary","sum"),  
    Average_Salary=("Salary","mean"),  
    Highest_Salary=("Salary","max")  
)
```

Output

Department	Total Salary	Average Salary	Highest Salary
------------	--------------	----------------	----------------

HR	165000	55000	60000
IT	225000	75000	80000
Sales	93000	46500	48000

Grouping Multiple Columns

Group by Department and City.

```
df.groupby(["Department", "City"])["Salary"].sum()
```

Output

Department	City	Salary
HR	Delhi	105000
HR	Mumbai	60000
IT	Delhi	75000
IT	Mumbai	150000
Sales	Pune	93000

Selecting Multiple Columns

```
df.groupby("Department")[["Salary", "Experience"]].mean()
```

Output

Department	Salary	Experience
HR	55000	4

IT	75000	5.3
Sales	46500	3

Using agg()

```
df.groupby("Department").agg({  
    "Salary":["sum","mean","max"],  
    "Experience":["mean","max"]  
})
```

Output

A multi-level column DataFrame is returned.

Using Named Aggregation

```
df.groupby("Department").agg(  
    Average_Salary=("Salary","mean"),  
    Maximum_Experience=("Experience","max")  
)
```

Filtering Groups

Suppose we only want departments where the average salary is greater than ₹50,000.

```
df.groupby("Department").filter(  
    lambda x: x["Salary"].mean() > 50000  
)
```

Output

Only IT and HR departments are returned.

Transform

Transform returns data with the same shape as the original DataFrame.

```
df["Average Salary"] = df.groupby("Department")["Salary"].transform("mean")
```

```
print(df)
```

Output

Employee	Department	Salary	Average Salary
Amit	IT	70000	75000
Rahul	HR	50000	55000
Priya	IT	80000	75000
...			

Applying Custom Functions

```
def salary_range(x):
```

```
    return x.max() - x.min()
```

```
df.groupby("Department")["Salary"].agg(salary_range)
```

Output

Department	Salary Range
HR	10000

IT	10000
Sales	3000

Sorting Grouped Results

```
df.groupby("Department")["Salary"].mean().sort_values(ascending=False)
```

Output

Departments sorted by average salary.

Resetting Index

```
result = df.groupby("Department")["Salary"].sum().reset_index()
```

```
print(result)
```

Output

Department	Salary
HR	165000
IT	225000
Sales	93000

Real-World Example 1

Sales Data

```
sales = {  
    "Product":["Laptop","Laptop","Mobile","Mobile","Tablet"],
```

```
"Sales":[50000,60000,30000,35000,25000]
}
```

```
df = pd.DataFrame(sales)
```

```
print(df.groupby("Product")["Sales"].sum())
```

Output

Product	Sales
Laptop	110000
Mobile	65000
Tablet	25000

Real-World Example 2

Student Marks

```
students = {
    "Class":["A","A","B","B","C"],
    "Marks":[80,90,70,75,88]
}
```

```
df = pd.DataFrame(students)
```

```
print(df.groupby("Class")["Marks"].mean())
```

Output

Class	Average Marks
-------	---------------

A 85

B 72.5

C 88

Difference Between `count()` and `size()`

<code>count()</code>	<code>size()</code>
Ignores missing values	Counts all rows
Works column-wise	Works group-wise
Returns non-null count	Returns total rows

Common Mistakes

Forgetting Aggregation

```
df.groupby("Department")
```

This only creates a GroupBy object.

Always apply an aggregation function such as:

```
.sum()
```

```
.mean()
```

```
.count()
```

Using Wrong Column

```
df.groupby("Department")["Salary"].mean()
```

Ensure the selected column exists and is appropriate for the aggregation.

Best Practices

- Use meaningful grouping columns.
 - Apply multiple aggregation functions with `agg()`.
 - Use `reset_index()` when you need a regular DataFrame.
 - Use `transform()` when you need results aligned with the original data.
 - Use named aggregations for clear and readable output.
-

Summary

In this chapter, you learned:

- What GroupBy is.
 - The Split–Apply–Combine concept.
 - Grouping by one or multiple columns.
 - Aggregation functions such as `sum()`, `mean()`, `count()`, `max()`, and `min()`.
 - Applying multiple aggregations with `agg()`.
 - Named aggregation.
 - Filtering grouped data.
 - Transforming grouped data.
 - Custom aggregation functions.
 - Real-world applications of grouping and aggregation.
-

Key Takeaways

- **Grouping** organizes data into meaningful categories for analysis.
- **Aggregation** summarizes grouped data using functions such as `sum()`, `mean()`, `count()`, `min()`, and `max()`.
- The **GroupBy** operation follows the **Split** → **Apply** → **Combine** approach, making it highly efficient for analyzing large datasets.
- Functions like `agg()`, `filter()`, and `transform()` provide flexibility for advanced data analysis tasks.
- Grouping and aggregation are essential skills for building reports, dashboards, business analytics, and machine learning preprocessing pipelines.